從後天八卦到十二地支：**Agentic Harness** 的 **Runtime Topology ——**以 **12 Agentic Harness Patterns** 重構 **AI Agent** 工程生命週期

——以 **12 Agentic Harness Patterns** 重構 **AI Agent** 工程生命週期

摘要

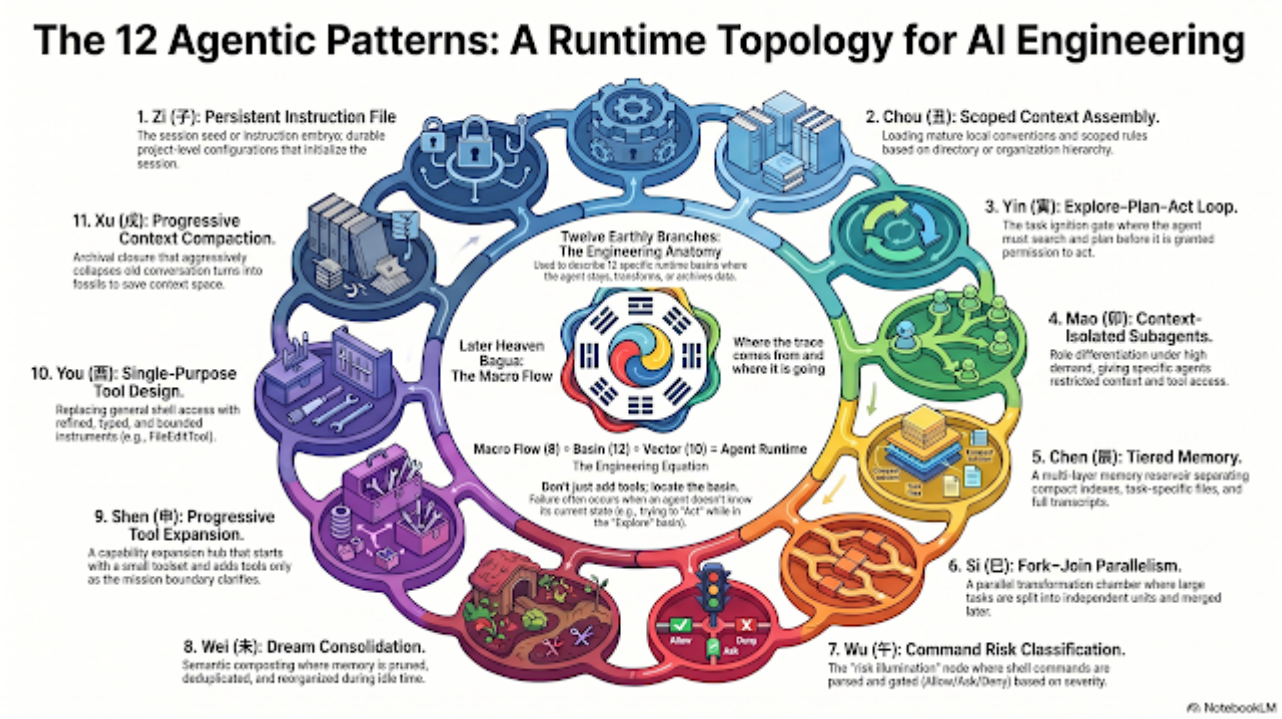
本文嘗試用一套工程化語言，重新理解 **12 Agentic Harness Patterns**。這 12 個 pattern 原本被描述為 production coding agent 的 reusable design patterns，分為 **Memory & Context**、**Workflow & Orchestration**、**Tools & Permissions**、**Automation** 四組；其核心不是單一 prompt 技巧，而是 agent runtime 如何保存記憶、組織工作流、限制工具權限、維持生命週期不變式。

本文主張：若用 後天八卦 來看，這套框架最自然地呈現為 agent runtime 的宏觀流勢圖；但若用 十二地支 來看，則能更深入地把 12 個 pattern 理解為 12 個 runtime basin，即 agent trace 在實際工作中會反覆進入、停留、轉化、壓縮、再生的工程場位。

因此，本文的核心結論是：

> 後天八卦適合描述 **agent runtime** 的流勢；十二地支適合描述 **agent runtime** 的工程解剖。





---

1. 問題提出：為什麼 12 Agentic Patterns 不只是技巧清單？

12 Agentic Harness Patterns 表面上像是一篇工程文章：它列出 persistent instruction file、scoped context assembly、tiered memory、dream consolidation、context compaction、explore-plan-act、subagents、fork-join、tool expansion、risk classification、single-purpose tools、lifecycle hooks 等設計方法。

但若細看，這些 pattern 並不是同一層的技巧。

有些 pattern 是 記憶結構，例如 Tiered Memory 將 compact index、topic-specific files、full transcripts 分成不同載入層級。

有些 pattern 是 工作流結構，例如 Explore–Plan–Act 將 agent 工作分成 explore、plan、act 三階段，並逐步增加 write permission。

有些 pattern 是 權限與工具結構，例如 Command Risk Classification 會在 shell command 執行前解析 verb、flags、target，並透過 allow / ask / deny 規則進行 gating。

最後，Deterministic Lifecycle Hooks 則指出：一些每次必須發生的行為，不應該靠模型記憶，而要放在 lifecycle hook 中，例如 PreToolUse、PostToolUse、SessionStart、CwdChanged。

這說明一件事：

> **12 Agentic Patterns** 其實是一套 **agent runtime** 的生命週期拓撲，而不只是 **12** 條工程建議。

---

2. 基礎語言：什麼是 runtime topology？

本文不假設讀者熟悉拓撲學。這裡先用工程語言定義。

在一般工程系統中，我們常問：

- 現在系統處於什麼狀態？
- 下一步可以去哪裡？
- 哪些狀態會困住系統？
- 哪些狀態可以分叉？
- 哪些狀態會封存記憶？
- 哪些狀態必須自動觸發？
- 哪些轉換會帶來風險？

這些問題其實就是拓撲問題。拓撲學不一定要先理解成高深數學；在工程上，它首先是一套關於連接、鄰近、邊界、轉換、閉環 的語言。

我們可以把一個 agent runtime 看成一個狀態空間：

$$\mathcal{M} = X \times C \times W \times U \times R \tag{2.1}$$

其中：

符號	工程含義
X	codebase / directory / project location
C	context and memory state
W	workflow phase
U	tool and permission state
R	risk / validation / lifecycle state

agent 的工作過程，就是一條 runtime trace：

$$p(\tau) \in \mathcal{M} \tag{2.2}$$

其中 $p(\tau)$ 是 agent 在某個 semantic time τ 中的 runtime state。

若用微分幾何語言，可把 agent 的推進寫成：

$$dp/d\tau = F(p; I, M, T, H) \tag{2.3}$$

其中：

符號	含義
I	instructions / rules
M	memory structure
T	tools and permissions
H	hooks / lifecycle invariants

這裡不需要把公式理解成真正可計算的物理方程；它只是說：agent 的下一步不是隨機的，而是被 instructions、memory、tools、hooks 等結構共同導引。

---

3. 後天八卦：宏觀 runtime flow map

在 SMFT 文件中，後天八卦被理解為語義 trace 經過撓場偏折後穩定下來的 attractor 分布；它不是新的張力起點，而是 collapse trace 在文化、記憶、語言、環境中被扭曲後的穩定落點。

換成 agent engineering 語言：

> 後天八卦適合描述 **agent runtime** 的大勢流轉。

它回答的是：

```
(3.1) trace 大體上從哪裡來，往哪裡去？
```

若用後天八卦來看 12 Agentic Patterns，大約可以得到這樣的宏觀流：

後天卦意	**Agent runtime** 意義
震	新任務啟動、agent 開始介入
巽	多路探索、分散掃描、branch 齊整
離	風險顯化、命令被照明、狀態相見
坤	工具／資源／規則進入可役使狀態
兌	interface、交換、輸出、使用者回饋
乾	壓力下的裁剪、核心化、不可省結構
坎	風險、陷落、命令危險、context failure
艮	止界、成熟規則、local convention 固化

這個框架非常自然，因為 12 Agentic Patterns 本身就是 agent 在真實工程場景中如何從記憶、流程、工具、權限、automation 逐步形成穩定運行的描述。附件也明確指出，這些 patterns 不是暫時產品特性，而是 agentic harness design 的 cornerstones，例如 memory tiering、context compaction、permission gating、lifecycle hooks。

但後天八卦有一個限制：它只有 8 個主節點，而附件有 12 個 patterns。若全部壓進八卦，會損失工程粒度。

因此，需要第二層：十二地支。

---

4. 十二地支：**runtime basin** 的工程解剖

在 SMFT 的干支文件中，地支不是月份標籤，而是語義動力學中的 attractor 張力幾何節點網絡。地支不只是標註時間，而是吸附 trace、封存記憶、誘發再現的場域。

更具體地說，地支的藏干可理解為 attractor basin 中封存的語義記憶層級：本氣最容易被 collapse 出來，中氣需要特定角度才激活，餘氣則是深層 semantic inertia。

這對工程系統非常重要。

因為一個 agent pattern 也不是單純動作，而是一個 runtime basin：

$$(4.1)\ B_i \subset \mathcal{M}$$

其中 B_i 是第 i 個 agent pattern 所代表的狀態盆地。

當 agent trace $p(\tau)$ 進入 B_i，它會受到該 pattern 的規則、記憶、權限、工具、壓縮方式或 hook 機制影響：

$$(4.2)\ p(\tau) \in B_i \Rightarrow p(\tau + \Delta\tau) \text{ 被 } B_i \text{ 的 local rules 導引}$$

這就是「十二地支」比「後天八卦」更有系統工程深度的原因：

> 後天八卦看大勢，十二地支看每個 runtime 場位如何工作。

---

5. 12 Agentic Patterns 的十二地支對應

以下是本文建議的 12 對應表。

地支	**Agentic Pattern**	工程本質
子	Persistent Instruction File	session seed / instruction embryo
丑	Scoped Context Assembly	mature local convention / scoped rule cave
寅	Explore–Plan–Act Loop	task ignition / read-plan-act gate
卯	Context-Isolated Subagents	role differentiation under new runtime demand
辰	Tiered Memory	multi-layer memory reservoir
巳	Fork–Join Parallelism	parallel transformation chamber
午	Command Risk Classification	risk illumination / visibility node
未	Dream Consolidation	semantic composting / idle memory fermentation
申	Progressive Tool Expansion	post-boundary capability expansion hub
酉	Single-Purpose Tool Design	refined metal instrument / typed bounded tool
戌	Progressive Context Compaction	archival closure / compressed trace fossil
亥	Deterministic Lifecycle Hooks	post-war seed kernel / lifecycle invariant

這個表不是術數判斷，而是工程拓撲表。它的意思不是「這個 pattern 等於某個命理符號」，而是說：

> 每個 **pattern** 都像一個 **runtime basin**，有自己的進入條件、主功能、內部記憶、輸出方式與下一步連接。

---

6. 逐支工程解釋

6.1 子：Persistent Instruction File

Persistent Instruction File 是每次 session 開始時自動載入的 durable project-level configuration，內含 build commands、test commands、architecture rules、naming conventions、coding standards。

這就是 agent 的「種子」。

```
(6.1) B子 = {p ∈ 𝓜 | p 由 durable instruction 初始化}
```

子不是顯性執行，而是 session 尚未展開前的最小規則胚胎。

6.2 丑：Scoped Context Assembly

Scoped Context Assembly 不是普通 context loading，而是根據 organization、user、project root、parent directories、child directories 等多層 scope 動態載入 instructions。

這像丑位的成熟封藏：不是新發明，而是不同目錄、不同子系統中已被時間篩選過的 local convention。

```
(6.2) B丑 = scoped local convention basin
```

它對應「成言乎艮」的工程版本：老系統、老模組、老業務線的成熟規矩，不必每次重建，而是按場域取出。

6.3 寅：Explore–Plan–Act Loop

Explore–Plan–Act 的本質是避免 agent 一開始就亂改 code。它要求 agent 先 read/search/map codebase，再 discuss plan，最後才 act。

這是寅：初動之門。

```
(6.3) B寅 = {read-only exploration → plan → controlled action}
```

寅不是大爆發，而是新任務進入可控行動的第一個門檻。

6.4 卯：Context-Isolated Subagents

Context-Isolated Subagents 讓不同 subagents 有自己的 context window、system prompt、restricted tool access；research agent 不能 edit code，planning agent 不能 execute commands。

這不是單純 branch，而是新局面下的角色分化。

```
(6.4) B卯 = role-differentiation basin
```

卯位的工程含義是：舊的一體 agent 不再足以應付新需求，系統開始分化出 specialized agents。

6.5 辰：Tiered Memory

Tiered Memory 是最明顯的「地支藏干」結構。它把 memory 分成 always-loaded compact index、topic-specific files、full transcripts on disk。

這與本氣／中氣／餘氣非常接近：

Memory tier	藏干式工程含義
compact index	本氣：常駐主記憶
topic files	中氣：按任務激活
full transcripts	餘氣：深層殘留，必要時 search

```
(6.5) B辰 = multi-layer recall basin
```

辰不是單純記憶，而是可再現、可搜索、可干擾 agent trace 的記憶水庫。

6.6 巳：Fork–Join Parallelism

Fork–Join Parallelism 會把大型任務拆成多個 independent units，spawn 多個 subagents，在 isolated git worktree 中平行工作，最後 merge。

巳的重點不是 fork，而是「高能轉化室」。

```
(6.6) B巳 = parallel transformation chamber
```

這裡有很強的「齊乎巽」味道：多路分散處理，但必須保持最後可 merge。若不能合流，parallelism 會變成混亂。

6.7 午：Command Risk Classification

Command Risk Classification 把 shell command 的潛在危險顯化：先解析 verb、flags、target，再用 allow / ask / deny rules gating。

這是午：風險被照亮。

```
(6.7) B午 = risk illumination basin
```

它與巳形成一段流：

```
(6.8) B巳 → B午 = parallel hidden work → visible classified risk
```

這正有「齊乎巽 → 相見乎離」之象：多路工作先被齊整，再進入可見、可審批、可分類的離火場。

6.8 未：Dream Consolidation

Dream Consolidation 在 idle time review、deduplicate、prune、reorganize memory；原文稱它像 agent state 的 garbage collection。

但它不是硬刪除，而是堆肥。

```
(6.9) B未 = semantic composting basin
```

未的工程意義是：將舊 trace、重複項、矛盾記憶轉化成更可用的養分。

6.9 申：Progressive Tool Expansion

Progressive Tool Expansion 從 small default toolset 開始，需要時才啟用 MCP tools、remote tools、custom skills，以避免一次給太多工具造成 selection problem。

這不是無邊界擴張，而是在任務、風險、邊界已較清楚後，逐步開放 capability。

```
(6.10) B申 = post-boundary capability expansion hub
```

它有「致役乎坤」之意：在已有邊界和任務分工後，工具被納入可役使狀態。

6.10 酉：Single-Purpose Tool Design

Single-Purpose Tool Design 把 general shell 操作改為 FileReadTool、FileEditTool、GrepTool、GlobTool 等 typed、bounded、permissioned tools。

這非常酉。

```
(6.11) B酉 = refined instrument basin
```

酉的工程本質是「專器專用」：工具不再是萬能 shell，而是被切割、打磨、定型、加上 validated inputs 與 clear boundaries。

6.11 戌：Progressive Context Compaction

Progressive Context Compaction 將 recent turns 保持 full detail，older turns 輕摘要，very old turns aggressive collapse。

這是戌：封口與壓縮。

```
(6.12) B戌 = archival closure basin
```

戌不是記憶再生，而是把舊 trace 壓縮成低解析度但可保存的 archive fossil。

6.12 亥：Deterministic Lifecycle Hooks

Lifecycle Hooks 指那些每次必須發生的行為，不應靠 prompt instruction，而要在 agent lifecycle 的特定點自動觸發；原文強調，anything that must happen every time belongs in a hook, not in an instruction。

亥不是普通 automation，而是「乾戰後的亥」。

在系統經過錯誤、遺忘、context 壓力、工具風險後，最後留下不可再省的 runtime invariant。

```
(6.13) B亥 = lifecycle invariant seed-kernel
```

它像經濟收縮後期企業只保留核心銷售／核心生產功能，也像植物在衰退後將能量集中到種子。

---

7. 後天八卦與十二地支的分工

現在可以清楚區分：

框架	問什麼問題	對 **agent engineering** 的用途
後天八卦	大勢如何流？	runtime flow map
中天八卦	當下做什麼任務？	task-state classifier
十二地支	trace 落入哪個工程場位？	lifecycle basin architecture
天干／五行十態	用什麼操作向量出手？	actuator primitives

SMFT 文件中已明確區分：後天八卦記錄的是「張力沉澱之地」，中天八卦描繪的是「張力流動之徑」；也就是從 collapse basin 到 collapse geodesic 的轉換。

本文進一步補充：

> 十二地支記錄的是 **runtime trace** 的十二個工程場位。

---

8. 工程化總模型

把三層結合，可得到：

$$\text{Agent Runtime} = \text{Flow}_8 \circ \text{Basin}_{12} \circ \text{Vector}_{10}$$

其中：

層	對應	工程角色
Flow8	後天八卦	大勢流轉
Basin12	十二地支	runtime state basins

層	對應	工程角色
Vector10	天干／五行十態	操作向量與工具動作

也可以寫成：

(8.2) $p(\tau + \Delta\tau) = \Phi_{支_i}(\Phi_{卦_j}(p(\tau), action_{干_k}))$

意思是：

1. agent 先處於某個宏觀 flow；
2. trace 落入某個 runtime basin；
3. 再由具體 operation vector 推動下一步。

工程上可翻譯為：

(8.3) `next_state = pattern_basin.apply( flow_context, operation )`

---

9. 為什麼這比單純 **pattern catalog** 更有用？

因為 catalog 只告訴你：

(9.1) 這裡有 12 個 pattern。

但 runtime topology 告訴你：

(9.2) 這 12 個 pattern 如何構成一個可循環、可壓縮、可分工、可治理、可再生的 agent 系統。

它可以帶來幾個工程設計原則。

9.1 不要只加工具，要先定位 **basin**

很多 agent 系統失敗，不是因為工具不夠，而是因為 agent trace 不知道自己在哪個 basin。

例如：

- 正在寅位，就應 explore-plan-act，不應亂寫 code；
- 正在午位，就應 classify risk，不應直接 execute；
- 正在戌位，就應 compact context，不應再無限延伸對話；
- 正在亥位，就應把 invariant behavior 做成 hook，而不是再寫 prompt reminder。

9.2 記憶不是一種東西，而是多個地支場位

memory 至少包含：

地支	記憶型態
子	seed instruction
丑	scoped convention
辰	tiered recall reservoir
未	consolidation compost

地支	記憶型態
戌	compressed archive
亥	lifecycle invariant

這比「long-term memory / short-term memory」的二分更細。

9.3 工具治理不是 **permission** 一件事

工具治理也至少有三層：

地支	工具治理
午	risk visibility
申	progressive capability expansion
酉	single-purpose tool refinement
亥	lifecycle hook invariant

這解釋了為什麼只做 permission gating 不夠。真正成熟的 agent harness 要同時有 risk classifier、tool expansion policy、typed tools、lifecycle hooks。

---

10. 結論

本文提出一個工程化轉譯：

> **12 Agentic Harness Patterns** 可以被理解為 **agent runtime** 的十二地支拓撲。

後天八卦提供宏觀 flow：它讓我們看見 agent runtime 如何從啟動、分化、顯化、承載、工具化、核心化、封存、再生中循環。

十二地支提供工程深度：它讓我們把 12 個 pattern 逐一理解成 runtime basin，而不是散亂技巧。

最終模型可壓縮為：

```
(10.1) 後天八卦 = agent runtime 的流勢圖

(10.2) 十二地支 = agent runtime 的結構解剖圖

(10.3) 12 Agentic Patterns = 十二個可工程實作的 runtime basin
```

更簡潔地說：

> 後天八卦讓我們看見 **agent** 的氣候；十二地支讓我們設計 **agent** 的器官。

這正是本文的核心命題：
若只用後天八卦，12 Agentic Patterns 會成為一張漂亮的大勢圖；
若加入十二地支，它才真正變成一套可工程化、可診斷、可實作、可擴展的 agent runtime topology。

Reference

12 Agentic Harness Patterns from Claude Code - The leaked source revealed how a production coding agent is actually assembled. Here are the design patterns behind it.
https://generativeprogrammer.com/p/12-agentic-harness-patterns-from

© 2026 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT 5.4, Google Gemini 3, NoteBookLM, X's Grok, Claude' Sonnet 4.6 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.